

Texture Synthesis using Generative Adversarial Networks

181975655

Artificial Intelligence

Abstract—In recent years, Generative Adversarial Networks (GANs) have undergone remarkable advancements. This research aims to leverage GAN techniques to innovate in the domain of texture synthesis. We present the Text2Texture GAN (T2TGAN), a novel approach in the domain of texture synthesis, tailored for generating textures directly from textual prompts. Envisioned as a valuable asset for artists, developers, professionals and laypersons within the 3D digital industry. T2TGAN is built upon the integration of state-of-the-art Natural Language Processing (NLP) techniques and GAN frameworks. T2TGAN aims to fulfill the gap present in texture synthesis by innovating on traditional texture synthesis methods. While the system's potential is palpable, the study encountered specific challenges, including overfitting, dataset limitations, and computational restrictions affecting the model's intricate design. Through robust quantitative and qualitative evaluations, we gather comprehensive insights into the system's competencies and identify possibilities that warrant improvement. Despite the identified challenges, the T2TGAN exemplifies a crucial step in the exploration of automated texture generation from textual prompts, presenting a significant contribution to the field of texture synthesis.

Keywords—GAN, Texture synthesis, Texture generation, Text2Texture

I. INTRODUCTION

The process of generating realistic and visually appealing textures, plays a vital role in computer graphics, virtual reality, and digital content creation. Creating textures is time-consuming and requires niche expertise skills to able to create high-quality work. This presents a challenge for individuals who work in the 3D graphics domain that require specific high-quality textures but do not possess robust texture creation skills. Texture synthesis refers to the process of algorithmically generating new textures that are perceptually similar to a given sample texture. Over the years, several approaches have been proposed for texture synthesis, and these can be broadly categorized into traditional methods and machine learning-based methods. Traditional methods for texture synthesis include patch-based algorithms, procedural methods, and statistical approaches. Patch-based algorithms synthesize textures by copying and stitching together small patches from the sample texture. While these methods can produce high-quality results, they often require careful patch selection and placement to avoid visible seams and artifacts. They are also very limiting in that they often fail to output specific textures which lead to many still preferring to create textures by hand. Deep learning has made significant progress in all image generation domains. These recent advancements in Generative Adversarial Networks (GAN) [1], offer exciting possibilities for enhancing texture synthesis capabilities. Despite these recent advancements, there is still insufficient literature and work on this topic. The most notable efforts in applying GANs to the realm of texture synthesis have focused on utilizing fundamental GAN techniques, rather than providing a

GAN that is able to provide textures based on specific inputs [3]. The primary aim of this research project is to implement a novel multi-modal text-to-image GAN named Text2Texture GAN (T2TGAN) that leverages state-of-the-art GAN and NLP techniques. The GAN will take a text description as input and output a texture consisting of an albedo (color) and a normal map. By introducing a prompt as an input, we seek to provide users with fine-grained control over the synthesized textures, enabling them to specify desired characteristics, styles, or visual attributes. This approach goes beyond traditional texture synthesis methods, allowing for more personalized and customizable texture generation. In the era of deep learning, our project aims to merge the realms of natural language processing and computer vision, combining text comprehension and image generation to create a robust texture synthesis model. The overarching goal of this research is to implement a GAN that leverages state-of-the-art techniques, utilizing textual prompts to offer users a novel and effective means of generating high-quality textures. This work strives to lessen the manual effort involved in conventional texture creation, opening up the possibility for more individuals to access top-tier textures, regardless of their experience or skill level.

II. RELATED WORK

A thematic approach is undertaken to review and discuss the existing research and methodologies related to implementation of texture synthesis using GAN.

A. Generative Adversarial Network

Generative Adversarial Network is a class of generative models that consist of two neural networks known as the *Generator* (G) and the *Discriminator* (D). The generator generates synthetic data samples using random noise vectors, while the discriminator aims to distinguish between real and fake samples. The two networks are trained simultaneously in an adversarial fashion, leading to the emergence of high-quality, realistic outputs. The GAN architecture comprises the generator and the discriminator. The generator takes random noise as input and transforms it into synthetic data that resembles

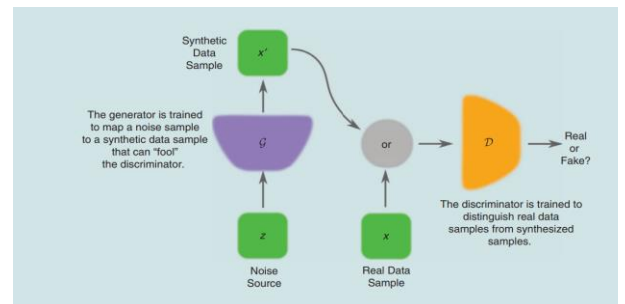


Fig. 1. High level GAN architecture

the training data. The discriminator, on the other hand, is trained to differentiate between real data samples from the training set and the generated sample [2] [4].

GANs are trained simultaneously, the training process can be described as a two-player minimax game. This adversarial process leads to an iterative training procedure, where the networks continuously improve until reaching a Nash equilibrium. The training objective of GANs is commonly defined using the adversarial loss, which encourages the generator to generate samples that fool the discriminator. This loss is typically formulated as a minimax problem, where the generator seeks to minimize the probability of the discriminator correctly classifying the generated samples, while the discriminator aims to maximize this probability. Equation 1 exhibits the minimax like training process. The probability that D determines the data generated by G is denoted as $D(G(z))$. To deceive D , the generator G aims to maximize the discriminator probability $D(G(z))$, while minimizing $\log(1 - D(G(z)))$ [4][5].

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (1)$$

B. Conditional Generative Adversarial Network

Conditional Generative Adversarial Network (cGAN) extends the traditional GAN framework by conditioning the generator and discriminator on additional information, such as class labels or other auxiliary data. This conditioning allows for the generation of specific samples based on the given input conditions instead of relying solely on random noise vectors. Additionally, this framework can also be used to learn a multi-modal model that allows multiple outputs given the same data [6]. In the context of our paper, this capability is crucial as it aligns with our goal of generating textures based on user-provided textual prompts or descriptions. The CGAN generator takes two inputs: a random noise vector and the auxiliary information. In the image generation domain, the goal of the generator is to learn the mapping from the joint input space to the image space such that the generated images closely match the target distribution of real images as illustrated in (2). We can perform the conditioning by feeding the additional extra information y into both the discriminator and generator as additional input layer.

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log D(x|y)] + E_{z \sim p_z(z)} [\log(1 - D(G(z|y)))] \quad (2)$$

C. Texture Synthesis and Image generation

Texture synthesis can be considered a sub-domain of image generation. The challenges highlighted in texture synthesis overlap with the challenges faced in the image generation domain. Consequently, the work of texture synthesis can be viewed as an extension of the image generation domain thus aptly integrates much of the techniques presented from image generation. One of the earliest works to apply GANs in the context of text-to-image generation was the StackGAN [7].

This two-stage model aims to generate high-resolution images from text descriptions, allowing for both the generation of images and the synthesis of textures with specific characteristics. The StackGAN works by generating low-resolution images in the first stage and refining them in the second stage to produce high-quality outputs. This hierarchical approach enables the model to capture both the overall structure and fine details present in the training data. StackGAN uses Word2Vec method to encode their prompts which can be a limitation given the advancement of recent NLP techniques [13].

The Self-Attention Generative Adversarial Network (SAGAN) [8] introduced self-attention mechanisms into the GAN framework, addressing some of the limitations of traditional convolutional layers. By incorporating self-attention, the SAGAN can model long-range dependencies between pixels, enhancing the quality of the synthesized images. This property is particularly valuable in texture synthesis, where it is often essential to capture and reproduce repeating patterns that may be present in the input data.

The use of self-attention mechanisms was further popularized by the Transformer architecture [9], which laid the groundwork for numerous developments in natural language processing (NLP) and computer vision. The Transformer model has been adapted for image generation, with models like the Vision Transformer (ViT) [10] demonstrating impressive results, outperforming the state-of-the-art convolutional networks. Residual networks (ResNets) have also played a significant role in advancing image synthesis and image generation [11]. These architectures alleviate the vanishing gradient problem inherent in traditional GANs by introducing skip connections, allowing the network to learn residual functions instead of the raw mapping. In the context of image generation, ResNets have been shown to effectively capture the intricate details present in images, improving the overall quality of the generated images.

D. Prompt Encoding

Prompt encoding is a vital technique of text-to-image synthesis using Generative Adversarial Networks (GANs). GANs primarily operate on numerical data representations thus they require data to be in numerical format for effective processing and generation. Consequently, it is imperative to transform plain text into a suitable numerical representation to facilitate meaningful communication between textual descriptions and the image generation process. This conversion must be carried out in an effective and purposeful manner, ensuring that the semantic content and essence of the original textual information are accurately translated into the numerical domain. A notable method is to utilize Word2Vec, often described as the cornerstone in natural language processing (NLP). Word2Vec is a word embedding technique that aims to represent words as dense vectors in a continuous vector space. Reference [12] introduced Word2Vec as two main architectures for representing words which were significantly more effective than one-hot encoding. Bidirectional Encoder Representations from Transformers (BERT) is a state-of-the-art pre-trained language model that represents a significant advancement in NLP techniques from the foundational

Word2Vec technique. BERT is utilized today in large language models. BERT utilizes two steps in the framework: pre-training and fine-tuning. In the pre-training phase, BERT is trained on massive text corpora to predict missing words in sentences. This unsupervised learning entails masking a certain percentage of words in a sentence and then training the model to predict these masked words based on the surrounding context. The bidirectional transformer architecture, with its attention mechanism, enables BERT to process each word in relation to all other words in the sentence, transcending the limitations of traditional models.

Once pre-trained, BERT can be fine-tuned for specific tasks using labelled data. In this supervised learning stage, BERT's powerful contextual embeddings are harnessed to address various natural language processing tasks such as text classification, named entity recognition, and question answering [13]. BERT has shown to increase performance when used in text-to-image GAN architectures [14].

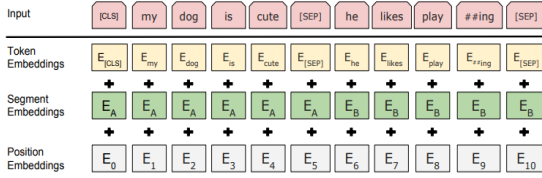


Fig. 2. BERT input representation

III. IMPLEMENTATION

A. Datasets

In this work, we utilized two distinct datasets available on HuggingFace [15]. The first dataset encompasses a collection of 1,000 color maps, each accompanied by a corresponding text description, commonly referred to as a "prompt". These prompts provide a brief textual representation of the associated color maps, which are known as albedo maps. Albedo maps capture the intrinsic color of an object without shadows or reflections, representing the diffuse reflectance of a material. The second dataset comprises a set of 1,000 normal maps, which serve as a complement to the albedo maps in the first dataset. Normal maps encode the surface normals of a texture, providing detailed information about the texture's geometric structure and how it interacts with light, enabling the creation of realistic three-dimensional effects. To facilitate the implementation, we merged these two datasets during the pre-processing phase, resulting in a consolidated dataset encompassing a prompt, albedo map, and corresponding normal map for each entry. Consequently, the final dataset contained a total of 852 examples as some examples were left out due to image corruption or incompatible prompts.

B. Data Pre-processing

The prompts were encoded using the BERT model. BERT operates by pre-training a transformer architecture on a large corpus of text to learn a vast array of contextual information. It differs from traditional word embedding techniques, such as Word2Vec or GloVe [16], as BERT considers both the left and right context of a word within a sentence. Consequently,

BERT captures a deeper and richer understanding of language semantics and syntactic relationships. Given our dataset comprises relatively short prompts, we opted to use the averaged embedding provided by BERT for text encoding. In BERT's pre-training phase, the average embedding is used as an aggregate representation of an entire input sequence for classification tasks. Therefore, this token contains a high-level contextual representation of the whole input sentence. By utilizing the average embeddings for our short prompts, we can effectively capture the entire semantic context of each description, thereby ensuring a coherent mapping between text and the generated textures.

C. Overview

The primary aim of T2TGAN is to synthesize two distinct images based on two inputs. **Fig.3** exhibits a high-level overview of this aim. The components of the system are as follows:

1) *Latent Noise Vector*: Serving as a source of randomness, this component ensures diversity in the generated images, enabling the GAN to produce a variety of outputs for the same textual prompt.

2) *Textual Prompt*: A descriptive text is processed and encoded using BERT to guide the image generation process, ensuring the output image is contextually relevant to the described content.

3) Outputs:

Color Image: A synthesized visual representation resonating with the provided textual prompt.

Normal Map: Alongside the color image, a corresponding normal map is produced, capturing the intricate surface details and orientations of the visual content.

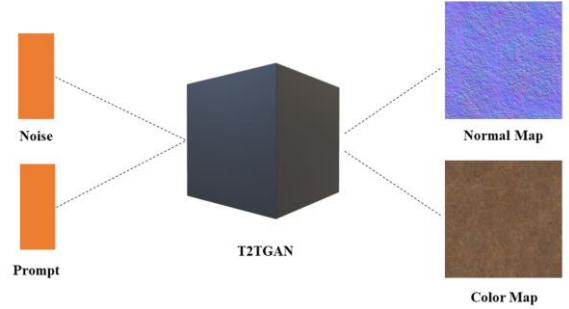


Fig. 3. T2TGAN Overview

D. Generator Details

We begin by introducing the noise vector into an Input layer. The noise vector, often referred to as the latent vector, is a crucial element in GANs and serves as the initial seed from

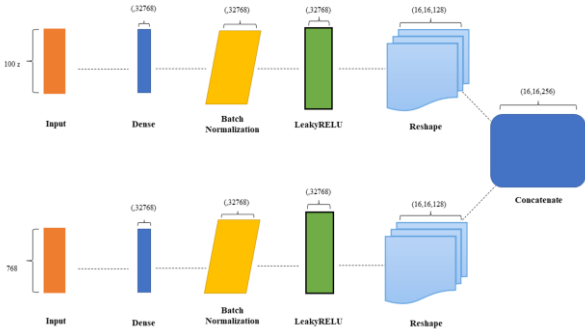


Fig. 4. Generator Input Processing

which the generator starts to create images. In addition to this we must condition our GAN on our encoded prompt. These inputs are then each passed separately through respective Dense layers with $128 * 16 * 16$ units. This layer performs a transformation which increases the representational power of the network. Following the Dense layer, a Batch Normalization layer is used. This layer normalizes the activations of the previous layer for each batch, i.e., transforms the activations to have zero mean and unit variance. This has been shown to improve the convergence speed of neural networks significantly [17]. Directly applying batch normalization to all layers however, resulted in sample oscillation and model instability. This was avoided by not applying batch normalization to the generator output layer and the discriminator input layer. After normalization, the activations are passed through a LeakyReLU activation function with an alpha value of 0.2. LeakyReLU, a variant of ReLU, allows a small, non-zero gradient when the unit is not active, which can help to alleviate the dying ReLU problem [2]. Finally, the activations are reshaped to a 16×16 feature map with 128 channels. This is done to prepare the activations for the convolutional layers that follow. The processed noise and prompt vectors are concatenated along the channel axis. This operation combines the information from both the noise and prompt vectors into a single tensor. This tensor becomes the input for the subsequent layers of the Generator. The concatenation of noise and prompt vectors allows the Generator to consider both the randomness introduced by the noise vector and the semantic information provided by the prompt vector when generating images as to not be prone to overfitting. This concatenation serves as a foundation for the subsequent convolutional, residual, and self-attention layers, which further refine and transform this input into the desired output textures.

Once the processed noise and prompt have been concatenated, we are primarily concerned with upscaling the input tensor, refining features, and producing the final albedo and normal maps. The architecture is designed based on the principles of convolutional neural networks and incorporates several techniques to enhance its performance. The architecture begins with a series of transposed convolutional layers, also known as "deconvolutional" layers. These layers perform the reverse operation of typical convolutional layers and serve to upscale the spatial dimensions of the input tensor. In this case, each

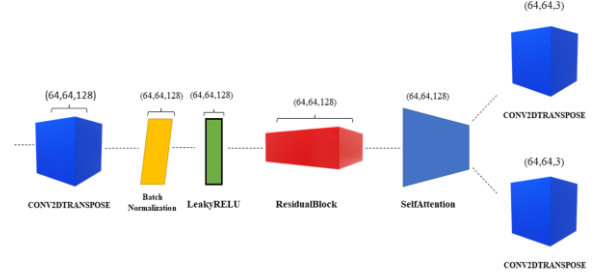


Fig. 5. Final Generator layers

transposed convolutional layer has a kernel size of 4×4 , a stride of 2x2, and padding set to 'same', which ensures that the spatial dimensions of the output tensor are exactly twice those of the input tensor. The choice of these parameters is intended to progressively enlarge the feature maps, allowing for a gradual increase in spatial resolution. Following each transposed convolutional layer, we employ batch normalization and the LeakyReLU activation function. In this case, the specific parameter alpha is set to 0.2 to ensure that the function is not too aggressive in negating negative values, preserving gradient information that may be useful for learning. After each activation function, a residual block is employed. Residual blocks are adapted from ResNets [11] to ease the training of very deep networks. The residual block will achieve this by introducing skip connections that bypass one or more layers. These connections allow our network to learn identity functions more easily, which in turn enables the stacking of more layers without hindering the learning process. In this architecture, residual blocks are used to increase the network's capacity without compromising its ease of training. Although not highlighted in Fig. 5, this process repeats twice before introducing the self-attention layer, allowing the network to increase the spatial resolution and extract features. Following this stage, a self-attention mechanism is employed. The self-attention mechanism enables the model to capture long-range dependencies within the data by computing attention scores between all pairs of positions in the input tensor. This technique has been proven to be extremely effective in capturing fine-grained details and contextual information in data. Finally, two separate transposed convolutional layers are used to produce the color and normal maps. These layers have three output channels and use the sigmoid activation function to ensure that the output values are within the range $[0, 1]$. This architecture is crafted to balance the requirements of upscaling, feature refinement, and the generation of high-quality textures. The combination of transposed convolutional layers, residual blocks, self-attention, and final output layers provides a robust framework for texture synthesis, capable of producing high-resolution, realistic textures that are consistent with the input prompts.

E. Discriminator Details

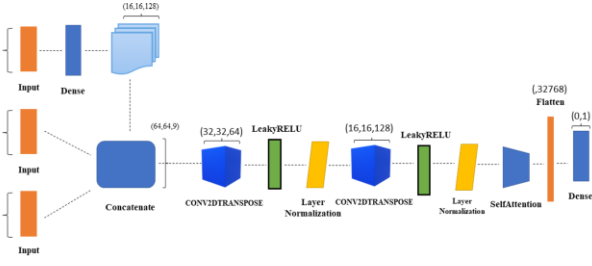


Fig. 6. Discriminator Architecture

The Discriminator in a Generative Adversarial Network (GAN) is responsible for distinguishing between real and generated images. In this T2TGAN, the Discriminator not only distinguishes between the two but also ensures that the generated images correspond to the input prompt. The architecture (see Fig.6) starts with three input layers, representing the color map, the normal map, and the prompt, respectively. The prompt, which is a fixed-length vector, undergoes a transformation to match the spatial dimensions of the image inputs. It is then concatenated along the channel dimension with the color and normal maps which is a design choice implemented from the conditional GANs architecture [6]. Following the concatenation, the combined tensor is subjected to a series of downsampling layers, each consisting of a convolutional layer followed by a LeakyReLU activation and LayerNormalization. The convolutional layers use a 3x3 kernel and a stride of 2x2, halving the spatial dimensions at each step. The LeakyReLU activation introduces non-linearity and allows for a small gradient when the unit is not active, which can prevent the gradients from becoming too sparse during training. LayerNormalization, unlike BatchNormalization, normalizes the features along the channel dimension, making it more suitable for small batch sizes and stable across a wider range of batch sizes. Next, the Self-Attention mechanism, inspired by the Transformer architecture [9], is applied to the tensor. This mechanism calculates attention scores for all pairs of positions in the input tensor, enabling the network to capture long-range dependencies and subtle interactions within the data. It has been shown to significantly enhance the performance of neural networks in image generation tasks.

Finally, the tensor is flattened and passed through a dense layer with a sigmoid activation, producing a single output value. This value represents the probability that the input images are real, as opposed to generated.

The Discriminator is designed to process both the color and normal maps, as well as the prompt, ensuring that the generated images are consistent with the input prompt. The use of LayerNormalization and the Self-Attention mechanism is motivated by their proven effectiveness in stabilizing training and capturing intricate details within the data. This architecture leverages these techniques to efficiently discriminate between real and generated images and to ensure that the generated textures correspond to the input prompts.

F. Training Process

The training process for the T2TGAN is conducted iteratively over a set number of epochs. During each epoch, the training dataset is divided into batches of a specified size, and the generator and discriminator are trained in a two-phase process.

1) Phase 1: Discriminator Training

The objective of the discriminator is to differentiate real images from generated ones. In this phase, the discriminator is updated to improve its classification capabilities. Using random noise vectors and input prompts, the generator produces a set of images that serve as the "fake" images for the discriminator. Simultaneously, real images from the training dataset are also utilized. The discriminator calculates two separate losses, one for real images and one for generated images, using the Binary Cross-Entropy loss function. This function is chosen due to its effectiveness in measuring the difference between two probability distributions: the output of the discriminator and the ground truth labels. The total discriminator loss is computed by summing the loss values for both real and generated images. This combined loss represents the overall error in the discriminator's classifications. The gradients of the total discriminator loss with respect to the discriminator's trainable parameters are computed using the backpropagation algorithm. An optimizer, specifically the AdamW optimizer with a learning rate of 0.0001, updates the discriminator's parameters following the computed gradients. AdamW optimizer was chosen due to GANs proclivity to overfit when working with limited data. In AdamW, weight decay is applied directly to the weights, which can lead to better generalization whilst also stabilizing the GAN training process [21].

2) Phase 2: Generator Training

The generator's objective is to produce realistic images that can fool the discriminator. During this phase, the generator is updated to enhance its image generation capabilities. New images are produced by the generator using random noise vectors and input prompts. The discriminator calculates a loss on the generated images, and the generator is updated to minimize the divergence between the discriminator's output and the real image labels. The gradients of the generator's loss with respect to the generator's trainable parameters are computed using backpropagation. The AdamW optimizer with a learning rate of 0.0001 updates the generator's parameters following the computed gradients. After each epoch, the generator and discriminator losses are recorded. This training process mirrors the methodology used in the original GAN work [1]-[3] and is adapted in the conditional GAN architectures [6].

G. Evaluation

To ascertain the efficacy of our GAN in translating textual prompts to visual outputs, it was imperative to employ robust evaluation metrics that include both quantitative and qualitative performance indicators. The assessment was divided into two primary dimensions: Fréchet Inception Distance (FID) for quantitative evaluation and human inspection for a more nuanced, qualitative appraisal.

1) 4.1 Frechet Inception Distance (FID)

The Fréchet Inception Distance (FID) is a widely accepted metric to evaluate the quality and diversity of generated images in the GAN landscape [18]. It gauges the similarity between the distribution of generated images and real images in the feature space determined by a pre-trained Inception network. A lower FID score suggests that the generated images are of higher quality and possess characteristics more akin to the real data distribution. Features from the intermediate layer of the Inception network were extracted for both the real and generated images.

These features were used to compute the mean and covariance for each distribution. The FID is then calculated using the Fréchet distance between the two Gaussian distributions defined by the extracted features. Our GAN's performance was benchmarked against this metric at periodic intervals during the training process. The aim was to monitor and ensure a consistent decline in FID values, indicating an enhancement in the quality and relevance of generated images over iterations.

2) 4.2 Human Inspection

While FID provides a quantitative measure of image quality, GAN outputs' true litmus test lies in their ability to resonate with human perception. It is inherently difficult to evaluate a GAN solely based on quantitative metrics. It is very possible that a text-to-image GAN can perform well on quantitative metrics but output very low-quality images. Hence, human inspection was incorporated as an additional evaluation methodology. A diverse panel of participants were included to offer their subjective evaluation on the outputs of T2TGAN. These participants were presented with the generator Colab file and were instructed on how to use it. Following five generated images. They were prompted to provide a qualitative assessment, focusing on aspects such as quality, correlation, and the potential use-case of T2TGAN in their work.

The feedback from human inspection complemented the quantitative results, offering insights into areas of potential avenues for refinement. More importantly, it ensured that the generated images were not only statistically aligned with real data distributions (as indicated by FID) but also visually and semantically coherent from a human perspective.

IV. EXPERIMENTS

In our pursuit to validate the robustness of T2TGAN, this experimentation section outlines the practical implementations, methodologies employed, and results amassed. The underpinnings of the experimental and evaluation setup are interpreted, providing insights into the computational efficiency and the resulting output of T2TGAN.

A. Experimental Setup

The experiments were executed on an Nvidia A100 GPU, which is recognized for its high computational power and is often a preferred choice for deep learning applications. Leveraging Google Colab's cloud-based platform, the training process could be executed seamlessly, ensuring consistent availability of resources. The model was built upon the TensorFlow framework, an open-source library that facilitates

the rapid construction and scalable training of deep learning models. All the scripts and methodologies were implemented using Python 3, which offers extensive support for AI applications and notebooks, further enhanced by its vast ecosystem of supporting libraries. In terms of data, the final dataset encompassed 852 examples. The images from this dataset were resized to a resolution of 64x64x3. This resizing was instrumental for two main reasons: first, to accelerate the training process by reducing the computational load; and second, to conform to the limitations posed by the computational environment offered by Google Colab. The GAN was trained for a cumulative duration of 3 hours, spanning 1000 epochs. Each epoch represents a complete forward and backward pass of all the training examples. The choice of epoch count was based on preliminary tests that aimed to strike a balance between training time and model performance. The optimization process of the network was driven by a modified version of the AdamW optimizer. AdamW, being an extension of the Adam optimizer, inherently includes weight decay regularization. However, our model underwent certain refinement iterations, making necessary adjustments in the learning rate and optimization parameters to achieve better convergence.

B. Quantitative Results

The quantitative results consist of FID scores and the losses of the respective generator and discriminator.

TABLE I. FID SCORES PER EPOCH

Epoch	FID
0	419.2862243652344
250	371.53692626953125
500	360.7720642089844
750	387.492431640625
1000	359.914631221

TABLE 1 exhibits the FID scores. At epoch 0 the FID was 419.286 serving as our initial score. This was expected as the model was yet to be optimized, and the generated images were presumably distant in quality and distribution from the real images. At epoch 250 there was a noticeable drop in the FID score to 371.537. This decrement signifies that the model was improving, and the generated images were more aligned with the actual dataset. Epoch 500 shows further improvement, with the FID score reducing to 360.772. This shows a continued trend of the model becoming more adept at generating realistic images. A slight increase in FID to 387.492 was recorded at Epoch 750. Such occasional spikes can be attributed to the adversarial nature of GANs, where the generator and discriminator are in a continuous tussle. It's also possible that certain challenging patterns in the dataset led to this temporary regression. At the final epoch 1000, the FID score at the conclusion of the training was 359.915, marking the lowest and thus best score. In summary, there seemed to be a progressive

reduction in FID scores however the overall results were sub-altern. In addition to FID scores, we display the loss graphs of all epochs.

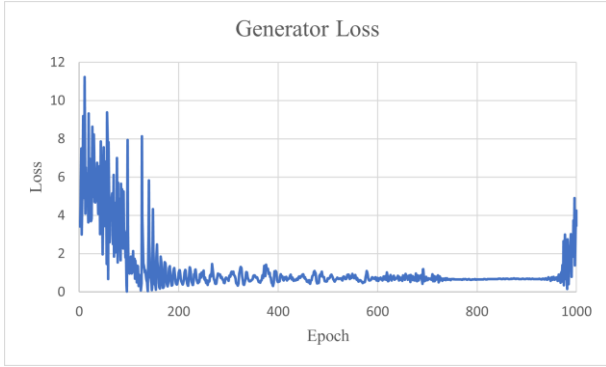


Fig. 7. Generator Loss Chart

Fig.7 exhibits the training loss of the generator, when plotted over epochs, provides a holistic view of the learning trajectory and model convergence. The high training loss at the onset indicates that the generator was initially producing images that the discriminator could easily distinguish from real images. This is typical for GANs at the beginning of the training, as the generator is yet to learn the features of the real data distribution. As training progresses, the consistent reduction in the loss signifies that the generator was becoming increasingly adept at creating images that are harder for the discriminator to distinguish from real ones. The rise in loss near the end of the training process could be a sign that the discriminator might have caught up and improved its capability to detect the nuances of the generator's creations, leading to an increased loss for the generator. Overall, the loss chart signified a well performing training process but the later epochs may provide some insight.

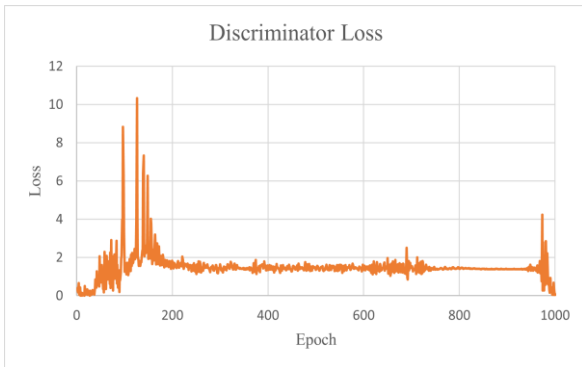


Fig. 8. Discriminator Loss Chart

The loss chart can be seen as an inverse process for the discriminator. In the initial stages it is very adept at detecting fake generated images due to the generator not learning the features of the data. As the training process continues, it is less adept at distinguishing fake images from real. Throughout most of the training, the loss was higher than that of the generator suggesting that the generator was able to fool the discriminator.

C. Qualitative Results

To complement our quantitative metrics, a user study was conducted to gauge the perceived quality and usability of the generated images. A group of 7 participants, representative of potential users, was invited to assess the outputs produced by the model. The objective of the qualitative evaluation is to offer a user-centric perspective on the model's performance. Responses were recorded using a Likert scale, ranging from 1 to 5.

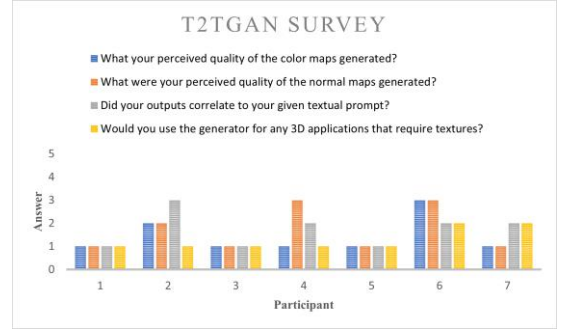


Fig. 9. Survey results

Fig.9 displays the answers of each four questions by each participant. There is a trend of unsatisfactory evaluations emerging from the users' responses. The generated color and normal maps were generally perceived as being of suboptimal quality. Furthermore, there was a discernible disconnect between the provided textual prompts and the corresponding generated images, indicating the model's challenge in faithfully translating textual prompts into visual outputs. Most tellingly, the likelihood of participants incorporating this service into their professional domain remained low. Such feedback underscores the need for further refinements in the model, both in terms of the underlying architecture and perhaps in the data it was trained on. Furthermore, answers above 3 on the Likert scale were not given for any question.

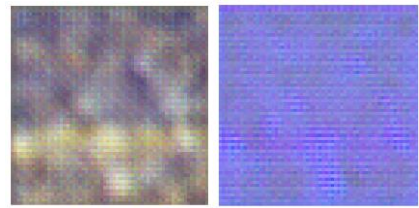


Fig. 10. A color map (left) and normal map(right) generated using T2TGAN

Fig.10 exhibits outputs generated from the prompt "bricks grey stone wall". Upon inspection, it is unclear whether there is a discrepancy between the prompt and the generated outputs, and the output being over-influenced by noise.

V. DISCUSSION

In evolving landscape of Generative Adversarial Networks, the results obtained from any novel model can provide insights, illuminating both its strengths and the areas necessitating refinement.

A. Analysis

Given our generator and discriminator loss charts, there was an anomaly. The rise in generator loss towards the conclusion of the training can be an indication of multiple factors:

1) *Adversarial Equilibrium:*

In the GAN framework, the generator and discriminator are in constant competition. The observed increase suggests that the discriminator might have caught up and improved its capability to detect the nuances of the generator's creations, leading to an increased loss for the generator [2]. This would be indicative of a training process that is not experiencing challenges.

2) *Overfitting:*

This increase however might also indicate that the generator started to overfit on the training data. This phenomenon can occur when the generator begins to memorize certain samples rather than understanding the underlying distribution [19].

3) *Learning Rate Dynamics:*

Given that learning rates and optimizer dynamics have profound impacts on GAN training, it's possible that the learning rate might have influenced this late-stage behaviour and must be adjusted.

B. Challenges and Limitations

Throughout the process of designing, training, and evaluating our GAN model, various challenges and limitations emerged, which have impacted the performance and the results obtained from the model.

1) *Risk of Overfitting:*

One consistent challenge faced when training neural network-based models, especially on smaller datasets such as in this case, is the potential risk of overfitting. The aforementioned problem of overfitting occurs when a model performs exceptionally well on the training data but struggles to generalize effectively to new, unseen data. This behaviour can be attributed to the model's tendency to capture noise or random fluctuations in the training data as opposed to the underlying distribution. In the context of GANs, overfitting can manifest in the generator memorizing specific samples rather than understanding the larger data distribution [1]-[2].

2) *Model Resizing and Reduced Complexity:*

The original design of our model aimed to output high-resolution textures of 512x512 pixels. However, due to constraints related to training time, the restrictions of Google Colab's paywall, and the computational cost involved, we had to resize the output to 64x64 as well as the training texture data to this texture as well. While this resizing expedited the training process, it inadvertently reduced the model's complexity. A multitude of layers that were planned and implemented in the GAN had to be removed. In the context of a downsized model, the benefits of architectural elements like

Residual blocks and skip connections might not have been fully leveraged. These elements, known for preserving spatial hierarchies and facilitating the training of deeper networks [11], may not have realized their full potential in a relatively smaller model framework. The resizing of the training data results in a loss of details by compressing the original resolution quite dramatically thus impacting the GANs ability to extract more features from the details of the textures.

3) *Dataset Size and Availability:*

One of the challenges encountered was the limited availability of a comprehensive texture dataset. Our final dataset comprised only 852 examples. The scarcity of texture datasets, especially those formatted with a color map, normal map, and a prompt, posed significant constraints. An ample and diverse dataset is fundamental for training robust GANs, and the limited dataset might have limited the model's ability to generate varied and high-quality outputs [20].

C. Future Work

Building upon the findings from our experiments and acknowledging the challenges and limitations encountered, we have identified several avenues for future research and development that can potentially enhance the performance and utility of the model. Given the observed potential for overfitting in our model, future endeavours should focus on techniques like dropout layers, and regularization to combat this issue. Incorporating these techniques can ensure that the model does not over-rely on specific patterns from the training data and achieves better generalization. One of the significant compromises made in the current research was the downsizing of the model's output from 512x512 to 64x64 due to computational constraints. Future work can aim to design and train a higher-resolution model, potentially facilitated by access to advanced computational infrastructure or distributed training methods to not lose the intricate details of the textures by resizing to a lower resolution. Additionally, the limited dataset of 852 examples was a significant constraint. Future research could benefit from sourcing, creating, or augmenting a more comprehensive texture dataset. Collaborations with industries or practitioners might be explored to gather more real-world texture examples. There are several large websites that hold tens-of-thousands of textures that can be put to use within this field. Furthermore, techniques like transfer learning, where knowledge from related tasks is leveraged, could be employed to alleviate the shortage of data. The use of advanced optimization techniques, such as adaptive learning rate methods or alternative loss functions, can be explored to further enhance the model's training dynamics. Periodic refinement based on real-world feedback can also be integral to the model's continued improvement. Moreover, given the unsatisfactory results from the initial human surveys, a more detailed feedback mechanism can be employed that would allow users to provide more insights into what they find lacking, offering clearer directions for model refinement. There also exists a need for comparison with other models. This was excluded in the current study due to no similar work being present, future endeavours can compare the proposed model with existing state-of-the-art texture generation models if available. While the current model provides a promising starting point, there is a plethora of methods and development opportunities that are to be explored. We are optimistic due to diagnosing a lot of the limitations. The future work involved in the hope that it could be

used to address the aforementioned challenges. We are confident that addressing these challenges will pave the way for a more robust and versatile texture generation model.

VI. CONCLUSION

The realm of Generative Adversarial Networks (GANs) has shown immense potential in numerous applications, notably in the generation of images. Our research delved into the design, implementation, and evaluation of a GAN aimed at synthesizing high-quality textures guided by textual prompts to generate color and normal maps. The detailing of the dataset, comprehensive architecture outlines of both the generator and discriminator, and the evaluation methodologies provided an in-depth understanding of the model's strengths and shortcomings. While our model showed promise in capturing the essence of the textual prompts and producing coherent textures, challenges such as potential overfitting, the constraint of reduced model complexity, and limitations in dataset size highlighted areas for improvement. The discussion section, reflecting upon these challenges and limitations, not only provided insights into the underlying causes but also paves the way for future explorations.

ACKNOWLEDGMENT

I would like to thank Carson Katri for making their texture dataset public. I am also thankful to my supervisor Professor Arumugam Nallanathan for his guidance and support throughout this journey. My sincere appreciation goes to the esteemed faculty at Queen Mary University of London, whose profound knowledge and insights have been instrumental in shaping my understanding in this domain. Additionally, I want to thank my beloved family for their constant encouragement.

REFERENCES

- [1] Pan, Z. et al. (2019) ‘Recent progress on Generative Adversarial Networks (Gans): A survey’, *IEEE Access*, 7, pp. 36322–36333. doi:10.1109/access.2019.2905015.
- [2] Goodfellow, I.J. et al. (2014) *Generative Adversarial Networks*, *arXiv.org*. Available at: <https://arxiv.org/abs/1406.2661>.
- [3] Wu, X., Xu, K. and Hall, P. (2017) ‘A survey of image synthesis and editing with generative Adversarial Networks’, *Tsinghua Science and Technology*, 22(6), pp. 660–674. doi:10.23919/tst.2017.8195348.
- [4] Creswell, A. et al. (2018) ‘Generative Adversarial Networks: An overview’, *IEEE Signal Processing Magazine*, 35(1), pp. 53–65. doi:10.1109/msp.2017.2765202.
- [5] Wang, K. et al. (2017) ‘Generative Adversarial Networks: Introduction and Outlook’, *IEEE/CAA Journal of Automatica Sinica*, 4(4), pp. 588–598. doi:10.1109/jas.2017.7510583.
- [6] Mirza, M., & Osindero, S. (2014). Conditional Generative Adversarial Nets. doi:10.48550/ARXIV.1411.1784
- [7] Xu, T. et al. (2017) StackGAN: Text to photo-realistic image synthesis with stacked generative adversarial networks, *arXiv.org*. Available at: <https://arxiv.org/abs/1612.03242v2>.
- [8] Zhang, H. et al. (2019) *Self-attention generative adversarial networks*, *arXiv.org*. Available at: <https://arxiv.org/abs/1805.08318>.
- [9] Vaswani, A. et al. (2017) *Attention is all you need*, *Advances in Neural Information Processing Systems*. Available at: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- [10] Dosovitskiy, A. et al. (2021) An image is worth 16x16 words: Transformers for image recognition at scale, *arXiv.org*. Available at: <https://arxiv.org/abs/2010.11929>.
- [11] He, K. et al. (2016) *Deep residual learning for image recognition*, *CVF Open Access*. Available at: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.
- [12] Mikolov, T. et al. (2013) *Efficient Estimation of Word Representations in Vector Space* Available at: <http://arxiv.org/abs/1301.3781>.
- [13] Devlin, J. et al. (2019) Bert: Pre-training of deep bidirectional Transformers for language understanding, *arXiv.org*. Available at: <https://arxiv.org/abs/1810.04805>.
- [14] Tsue, T., Sen, S. and Li, J. (2020) *Cycle Text-to-Image GAN with BERT*, *arXiv.org*. Available at: <https://arxiv.org/abs/2003.12137v1>.
- [15] *Dream-textures/textures-color-1K · datasets at hugging face* (no date) *HuggingFace*. Available at: <https://huggingface.co/datasets/dream-textures/textures-color-1k> (Accessed: July 2023).
- [16] Pennington, J., Socher, R. and Manning, C. (2014) “Glove: Global Vectors for Word Representation,” *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Available at: <https://doi.org/10.3115/v1/d14-1162>.
- [17] Ioffe, S. and Szegedy, C. (2015) Batch normalization: Accelerating deep network training by reducing internal covariate shift, *arXiv.org*. Available at: <https://arxiv.org/abs/1502.03167>.
- [18] Szegedy, C. et al. (2016) ‘Rethinking the inception architecture for computer vision’, *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. doi:10.1109/cvpr.2016.308.
- [19] Arjovsky, M., Chintala, S. and Bottou, L. (2017) *Wasserstein Gan*, *arXiv.org*. Available at: <https://arxiv.org/abs/1701.07875>.
- [20] Borji, A. (2019) ‘Pros and cons of gan evaluation measures’, *Computer Vision and Image Understanding*, 179, pp. 41–65. doi:10.1016/j.cviu.2018.10.009.
- [21] Loshchilov, I. and Hutter, F., 2019. Decoupled weight decay regularization. In *Proceedings of the International Conference on Learning Representations (ICLR)*.